Lesson 14: Raising the Stakes

TIC-80

VIDEO GAME CREATION

This week: later waves get genuinely harder - more enemies, moving faster - using the exact wave variable you've had since Lesson 11.

Open your own [YourName]Game project from Lesson 13 - don't start a new one.

Tip: no new assets this week - difficulty scaling reuses your exact existing enemy sprites and spawn logic.

Setup

1. Open your [YourName]Game project from Lesson 13.
2. No new globals needed for Level 1 - Level 2 will add a speed field to each enemy's table entry.

Level 1 - Minimal (MVP)

Each wave spawns one more enemy than the last (5, 6, 7). Speed stays the same for now - that's Level 2.

3. In spawn_wave(), change the fixed loop bound of 5 into 4 + wave (code below) - wave 1 spawns 5, wave 2 spawns 6, wave 3 spawns 7.
4. Wherever enemies_remaining gets set to 5 for a new wave, change that to 4 + wave too - these two places must always agree.
5. Press Esc, type run, press Enter, then play through a full round - wave 2 should visibly spawn one more enemy than wave 1, and wave 3 one more than that.

```
function spawn_wave()
 enemies = {}
 local count = 4 + wave
 for i=1,count do
  table.insert(enemies, {x=..., y=..., flash=nil})
 end
 enemies_remaining = count
end
```

Level 2 - Complete (Alpha/Beta)

Enemy speed ALSO increases each wave, on top of the growing count. Restarting correctly returns everything to wave 1's original values.

- In spawn_wave(), add a speed field to each new enemy entry: speed = 1 + (wave - 1) * 0.5.
- In your enemy-movement code, change the fixed downward increment to use e.speed instead of a literal number.
- Restart mid-game and confirm both count and speed return correctly to wave 1's values.
- Play a full round and confirm wave 3 is noticeably busier and faster than wave 1, but still fair.

Level 3 - Refined - Choose ONE Stretch Feature

Pick exactly one of the five ideas below. These are intentionally open-ended - a good-faith attempt at the general idea is a complete Level 3 finish, not a fully polished feature.

- Destructible shields/barriers: a few simple blocks placed between the player and the enemies that disappear (or shrink) after absorbing a hit or two - reuses your existing collision-detection and hit-reaction patterns from Lessons 4 and 13, just on a new kind of object.
- A power-up: an item that occasionally drops or spawns, which the player collects to temporarily change how shooting works (faster fire rate, multiple bullets at once, or a brief shield) - reuses your existing timer/cooldown pattern from Lesson 3.
- A bonus enemy: a rare, faster-moving, or oddly-patterned enemy worth extra points, spawned occasionally alongside a normal wave - reuses this week's own speed/count scaling ideas, just for one special case instead of the whole wave.
- Formation-marching movement: instead of each enemy independently drifting down (this course's approach since Lesson 4), have the whole wave move together as a group and occasionally step sideways - a genuinely different movement pattern, inspired by classic arcade shooters.
- A title screen and/or game-over screen: a proper opening screen (press a key to start) and/or a dedicated game-over screen with a final score, instead of just text overlaid on the gameplay - reuses your existing game_active flag and UI tools from Lessons 9-10. TIC-80's map() function (for drawing a background) is a natural fit for a simple title screen.

If Something's Wrong

- Every wave still spawns exactly 5: confirm the loop bound actually uses (4 + wave) and not a leftover literal 5.
- "You Win!" fires early or never: confirm enemies_remaining is set using the SAME formula everywhere it's set.
- Speed doesn't change between waves: confirm each enemy's speed field is being set INSIDE spawn_wave(), and that your movement code actually reads e.speed instead of a leftover literal.

Want More?

Student Resource List > TIC-80 section, or github.com/nesbox/TIC-80/wiki